Constraint-Based Approach for Automatic Hinting of Digital Typefaces

ARIEL SHAMIR
The Interdisciplinary Center, Herzelia, Israel

The rasterization process of characters from digital outline fonts to bitmaps on displays must include additional information in the form of *hints* beside the shape of characters in order to produce high quality bitmaps. Hints describe constraints on sizes and shapes inside characters and across the font that should be preserved during rasterization. We describe a novel, fast and fully automatic method for adding those *hints* to characters. The method is based on identifying hinting situations inside characters. It includes gathering global font information and linking it to characters, defining a set of constraints, sorting them, and converting them to hints in any known hinting technology (PostScript, TrueType or other). Our scheme is general enough to be applied on any language and on complex scripts such as Chinese Japanese and Korean. Although still inferior to expert manual hinting, our method produces high quality bitmaps which approach this goal. The method can also be used as a solid base for further hinting refinements done manually.

Categories and Subject Descriptors: I.3.3 [**Computer Graphics**]: Picture/Image Generation; I.3.5 [**Computer Graphics**]: Computational Geometry and Object Modeling; I.7.4 [**Text Processing**]: Electronic Publishing

General Terms: Algorithms, Design, Languages

Additional Key Words and Phrases: Digital typography, fonts, hinting, geometric constraints

1. INTRODUCTION

Character bitmaps are the visual representation of text in most output devices including both printers and displays. The demand for high quality bitmaps is increasing with the spread of small display devices such as PDAs, phones and e-books. This means any digital typeface (font) model must support a conversion procedure to bitmaps. Outline fonts are the most prevailing digital typeface models today. The newly announced OpenType universal font format [Adobe Systems Inc. 2000; Microsoft Corporation 2001] actually merges two distinct standards for digital typefaces: the TrueType [Microsoft Corporation 1990] and PostScript [Adobe Systems Inc. 1990] technologies. The characters in an outline font are represented by the boundary of their shapes, using some form of mathematical spline such as bézier spline (PostScript) or B-spline (TrueType). Although they are referred to as “outline fonts”, they are complex entities including more than just the outline boundaries of the character shapes. In fact, one can view digital typefaces not only as character outlines, but also as a type of procedure (e.g. a computer program), which converts the outlines into bitmaps. Upon request for a specific character from the font, the system calls a special program called *rasterizer* to execute this character procedure and create a bitmap image in the desired size. In order to create high quality bitmaps, rasterization requires *hints*,

Author's address: The Interdisciplinary Center, Herzliya, P.O. Box 167, Herzliya, 46150 Israel; email: arik@idc.ac.il.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or direct commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 1515 Broadway, New York, NY 10036 USA, fax: +1 (212) 869-0481, or permissions@acm.org.
© 2003 ACM 0730-0301/03/0400-0131 $5.00

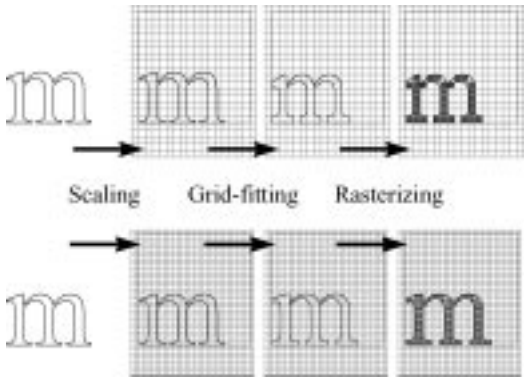


Fig. 1. Converting a character outline to a bitmap in a desired size includes scaling, grid-fitting and rasterizing. First, scaling is performed by imposing a grid on the outline (note that finer grid will result in a larger bitmap image, since contrary to this illustration, the pixel size will be constant), then grid fitting is performed according to predefined hints, and finally, the actual bitmap pixels are colored according to the rasterization algorithm (black and white in these examples).



Fig. 2. Some examples of typographic features.

which contain additional information regarding the typographic characteristics of different shape parts in the font. Defining and inserting hints into the characters of a font can be an extremely difficult and time-consuming manual process.

Rasterization of poor quality bitmaps in the absence of hints (see examples in Figure 16 and Figure 17), is mainly caused by the fact that rounding errors occur while scaling the font to the desired size. Scaling is performed by converting the outline font units, called units per em or UPM, to the bitmap pixel units, called points per em or PPM (see *scaling* in Figure 1). An excellent explanation of this problem can be found in Stamm [1998a]. Better results are achieved by deforming the outline to fit the grid lines in the desired size before rasterization (see *grid-fitting* in Figure 1). Grid fitting requires hinting information describing constraints on sizes and shapes that should be preserved during the deformation of the outline. The goal of hinting is therefore to maintain certain relations inside the shape of the characters while grid-fitting them.

The two major hinting technologies associated with the two major standards of outline fonts take a different approach to hinting. In PostScript fonts, the hints are mainly declarative; different parts of the characters, for example, typographic features such as bars, stems, arcs (see Figure 2) are marked with hints. The PostScript rasterizer itself is responsible for interpreting these hints and converting

them into grid-fitting constraints. This means that possible improvements to the rasterizer can affect the resulting bitmaps for all fonts using this rasterizer. However, this also means that the designer has limited control over the end result of her typeface. TrueType hints are procedural and allow much greater control over the resulting bitmaps up to the level of a single pixel. The hints are actually small programs written in a specialized assembly language, and stored inside the font model.

In both technologies hinting of characters in a font involves local (character) as well as global (font) considerations. Consider for example, the lowercase 'm' of Times Roman font from Figure 1. Grid-fitting constraints require that each of the three stems of the letter should have an equal width, and that the space between them (called counters) should be uniform. They also require that the joints of the arcs to the stems should be at least one pixel wide, that the serifs should have equal length extending to the left and right of the stem, and that all three occurrences should appear similar. Also, the height of the letter should be the same as the global height of all round letters such as 'n' or 'r'. The letter should sit on the font baseline and should not extend outside of its box to the right or the left.

Although to date many hinting techniques have been developed (Section 2), there is a set of sub-tasks shared by all of them:

Identifying hinting situations. Every hinting process relies on identifying special situations in the shape of individual characters that need to be addressed while grid fitting the outline [Karow 1994]. As seen in the 'm' example above, the bars and stems of characters need to preserve a uniform width and also maintain a minimum width in order to maintain its characteristic shape; the height of a character must be preserved; the character should lay on the baseline, and more. Identifying these special situations is often a matter of typographic skills and understanding.

Handling of global sizes. In order to preserve conformity and uniform appearance across the font the local hinting situations need to be linked to global font sizes. The character height might be linked according to its type as straight-character height (x-height) or round-character height (o-height). Bars and stems might be linked to a characteristic font width of such features found in many characters. These global font sizes can be defined by the designer or found by examining all characters in the font and gathering statistics.

Prioritizing. Hinters often encounter a situation where a full set of grid-fitting constraints in a character may impose contradictions or inconsistencies. For example, if one would like to preserve the total width of an 'm' character, one can constrain the right-hand side of the character to be a certain distance from the left-hand side. However, one would also like the three stems to have equal widths and the two counters to have equal widths in accordance with the stem widths. However, if the whole character width is already pre-determined it may be impossible to satisfy the second requirement. In such cases, prioritizing the different constraints and solving such inconsistencies, whether implicitly or explicitly, is also a part of any hinting process.

Translating. Lastly, if the hinting constraints were not created as low level hints, there is a need to translate them into the language of the specific font-technology in use (PostScript or TrueType or other).

The whole process of hinting can become extremely tedious and time consuming if done all manually. Furthermore, mastering this process involves expertise in typographic, esthetic as well as technical aspects. Previous hinting techniques performed the sub-tasks mentioned above mostly manually. Our scheme manages to perform them in a fully automatic manner. A hinting system based on this scheme uses a small set of tuning parameters (e.g. for feature extraction and application of specific hints) and automatically generates hinted outline fonts in PostScript or TrueType format. The system has been tested on hundreds of fonts and in various scripts. Although we do not claim our hinting results are comparable to manual hinting by an expert, our scheme gives good results especially in complex

languages such as Chinese or Japanese where manual hinting is almost impossible. Furthermore, these results can be used as an advanced starting point for manual hinting refinements, greatly reducing the time and effort of creating high quality fonts.

We shall start in Section 2 with a review of previous schemes for hinting. We present the details of our scheme in Section 3 and in Section 4 we discuss the translation to TrueType and PostScript hints. Section 5 shows some results and Section 6 summarizes.

2. PREVIOUS WORK

Hinting techniques can be classified into three categories: direct manual, interactive visual and semi-automatic. There are hinting tools that may use a mixture of these techniques, but generally concentrate on one.

The first PostScript and TrueType fonts were directly hinted by manually inserting the low level directives into each character representation. In PostScript this meant placing the declarations of features at specific positions along the contours of the outline, noting for instance their widths and heights. In TrueType this meant writing assembly-like programs to be executed for each character. However, procedures could not be continued for the large number of typeface libraries existing today, especially not for complex scripts such as Japanese, Chinese, and Korean. In such scripts, there are thousands of characters in each font, and each character is much more complex in shape than in roman fonts.

Since typefaces are mostly produced in commercial companies, most of the hinting methods are considered proprietary to those companies (Adobe, ITC, Letraset, Monotype, Bitsteam, Linotype and more). However, the solutions to the tedious manual hinting procedure are either visual or programmable. By using visual aids and interactivity, hinting becomes a more tolerable task: the points and character outlines are displayed, along with font metrics and heights, and the hinter can evaluate the effects of hinting by examining the bitmaps created after rasterization [Pyrus North America 2001; Microsoft Corporation 2000]. Furthermore, similar to the trend in general programming languages, higher-level and more declarative languages were created for hinting. This further simplified the task for the designer/hinter, but required some automatic translation to TrueType or PostScript low-level languages.

Both the visual and the programmable schemes are well illustrated by Microsoft's Visual TrueType package [Stamm 1998b; Microsoft Corporation 2000]. Visual TrueType Talk is a high level language for expressing relations between points inside the character. Since it is mainly targeted at TrueType hinting, each character holds a small VTT-Talk program. A visual editor is also available that allows the user to insert and edit some VTT commands visually and connect different character parts interactively. The visual symbols are converted to VTT-Talk commands, which in turn are compiled later into the TrueType hinting language. Although such visual and interactive aids make the hinting process simpler, the process is still essentially manual, and it still takes a very long time to create high quality hinted fonts.

To accommodate some of this, semi-automatic techniques of copying previously defined manual hints to new fonts are used. In Bétrisey and Hersch [1989] and Hersch and Bétrisey [1991] semi-automation is proposed for the hinting of Roman fonts. This method is based on manually defining a generalized model for each character topology. These model characters contain manually inserted hinting information. During the hinting of a new font, the automation process tries to match real characters to the models. If a match is found, the hints are carried over from the models to the matched parts of the outline. Later, this work motivated Zongker et al. [2000] to neglect the use of a previously defined generalized model. Instead, a matching algorithm between two different outlines fonts is used and the hints are carried from a previously manually-hinted outline to a new un-hinted one. Both techniques were tested mainly with simple topology Roman fonts, they are highly dependent on the arrangements, order and existence

of specific points on the contours of the character outlines, and work only when the matched characters have very similar topological shapes. The vast variety of typeface designs illustrate that these types of techniques have limited usability. Furthermore they are practically impossible for oriental typefaces where the characters are much more complex: building a generalized model and performing character matching are extremely difficult. Lastly, both techniques do not give any solution for the need to find and match global font sizes. In fact, in Zongker et al. [2000] if the global referred size differs too much in a matched hint, the whole hint is dropped.

Earlier attempts for automatic hinting techniques concentrated either on simple scripts (Roman fonts in Andler [1990]) or local hinting situations (vertical and horizontal stems in [Hertz and Hersch 1994]). There is a real need for automatic methods to help detect and define the dependencies between character parts and features that do not depend on prior knowledge about the shape and topology of the characters. There is need for an automatic method to extract global font sizes, freeing the hinter from tedious tasks such as measurements and statistics gathering. In this work we present an automatic hinting procedure for fonts that is based on the fact that most grid-fitting constraints are local in nature and are connected to some typographic feature. A feature extraction algorithm is employed, followed by comparison and gathering of statistics from all characters in the font. Finally, both features and statistics are used to create hints applicable to any hinting format.

3. AUTOMATIC HINTING

3.1 Overview

Our automatic hinting scheme is composed of five main stages:

(1) Identifying hinting situations

(2) Gathering font statistics

(3) Constructing a points relation graph

(4) Ordering and resolving graph inconsistencies

(5) Translating to a specific hinting language

The first task of any hinting process is finding the places inside each character where grid-fitting constraints should be imposed to preserve its shape. We call these places *hinting situations* (see Karow [1994, 1989]). Most hinting situations are local in nature and are connected to some typographic feature such as stem, bar, slant, joint and serif. In our scheme we exploit this fact and utilize a feature extraction algorithm defined in [Shamir and Rappoport 1996] in order to automatically find the hinting situations. Once the features are found and added to the characters, the second stage of gathering global font statistics can also be accomplished. These sizes will be used later in regularizing the font appearance by linking local constraints to the global sizes. Next, the constraints between high-level character features should be converted to relations between points. The reason for this is that the representation of characters in most font technologies is outline based and points are the basic entities one can address in all hinting languages.

It can be seen that even after gathering the hints, not all constraints can be preserved simultaneously at all bitmap resolutions (note e.g. the serifs in Figure 1). This means that any hinting process, be it manual or automatic, must handle difficulties not only of finding and defining the grid-fitting constraints, but also of ordering them, prioritizing them and preventing inconsistencies. This process is called *graph inconsistencies resolution* and the need for it is greatly enhanced in Far-eastern scripts where characters are much more complex (see Figures 17, 19, 20).

The last stage of automatic hinting, namely translating to hints, is highly dependent on the specific font-technology hinting language in use. However, since all technologies share the same basic set of relations, we use our well-constrained point graph and translate it according to any additional language requirements.

The rest of this section is composed as follows. In Section 3.2 we define the basic geometric constraints used in the process of hinting; mainly expressing relations between points on the outline of characters. Section 3.3 describes typographic feature extraction algorithms and the connection to hinting situations. Section 3.4 deals with gathering global font size statistics. Section 3.5 describes how we build the constraint graph and Section 3.6 how we resolve graph inconsistencies.

3.2 Constraints and Point Relations

As stated earlier, the goal of hinting is to create instructions modifying the outline in order to enforce at rasterization time certain relationships on and between the rasterized character sub-shapes. A most powerful tool used to express relations is the use of geometric constraints. The position and orientation of any geometric object is governed by its degrees of freedom (e.g. for translations and rotations). Geometric constraints are used to define relations between some objects by limiting their combined degrees of freedom. The simplest possible entity to be constrained inside a character is the point, and in fact most hinting mechanisms define hinting using constraints on points. A point in two-dimensional space has two degrees of freedom; hence each point can have at most two constraints. For example, a point p can be constrained to be in a specific distance d from a given line l; this limits p to lie on a line parallel to l and in distance d. Each constraint reduces the number of degrees of freedom of an object. The difference between the original number of degrees of freedom and their number after application of the constraint is called the *valency* of the constraint. The valency of the line-distance constraint on a point is 1.

A system of constraints can be represented by a hyper-graph whose nodes represent geometric objects and edges denote constraints between objects. Let f be the number of initial degrees of freedom of a node, and v be the sum of valancies of edges incident at the node. If $v > f$ the node is called *over-constrained*, if $v < f$ the node is *under-constrained*, and if $v = f$ the node is *well-constrained*. In general, under-constrained systems can have an infinite number of solutions, over-constrained systems are not solvable, and a well-constrained system possesses a finite number of solutions but it may possess none (for example, constraining a point to be a certain distance from two other points can have zero, one or two solutions depending on the value of the distances and locations of the points).

Solving a system of constraints is a very difficult problem [Bouma et al. 1995; Kramer 1992]; in fact, there is no general method that can always provide a solution. In the case of models for typefaces there are several additional requirements on the constraint system. First, a solution must exist in all cases; otherwise the typeface would be 'missing' some letters. Second, only a single solution must exist in all cases, otherwise there could be an undetermined number of designs for the same letter. Last, in this paper we convert the constraint system into native hinting languages such as TrueType and PostScript. This means that solving the constraint system is done offline. However, if constraints are used as part of the rasterization process, then the time to solve the constraint system of every letter must also be bound in real time limits. Otherwise, the user would have to wait until each letter is drawn, which is unacceptable in practice.

The geometric objects used to define the contours of the shape of characters in outline fonts are geometric splines and straight lines. These in turn are defined by a set of discrete points. Therefore, the ability to express geometric constraints in such a representation is mostly limited to points relations. Points have two degrees of freedom: two coordinates in Cartesian coordinates or a direction and a distance in polar coordinates. A constraint on a point can limit one or two degrees of freedom. The

simplest possible point-to-point(s) relations can be classified according to the number of constraining points (zero, one or two points constraining the position of one point):

—**Abs:** Point-absolute-value (**PAbs:** if the value v is a global parameter) is defined by:

$$p_0 = v$$

This type constraint can be meaningful in one of two situations:

(1) If the absolute value v is a point, this means v is constraining the two degrees of freedom of p_0.
(2) If v is a scalar we must measure the value of p_0 in a specific direction (project it onto a specific vector). The constraint then limits only one degree of freedom of p_0.

In the second case, the directions most commonly used for p_0 projection are the X or Y coordinate directions, which represent respectively setting the X or the Y coordinate of p_0 to a specific value or to a global parameter (i.e. **XAbs:** $p_0^x = v$ or **YAbs:** $p_0^y = v$).

—**Dist:** Point-distance (**PDist:** if the distance d is a global parameter) is defined by:

$$p_0 = p_1 + d$$

This constraint has one constraining point (p_1), and actually constrains p_0 to lay on a circle. However, when the distance between the points is measured as a signed distance in a given direction (projected onto a vector), it will limit p_0 to a line, constraining one degree of freedom of p_0. Again, if we choose the X or Y direction this constraint would become **XDist:** $p_0^x = p_1^x + d$ or **YDist:** $p_0^y = p_1^y + d$.

—**Ratio:** Point-ratio (**PRatio:** if the ratio is a global parameter) is defined by the equation:

$$p_0 = t \cdot p_1 + s \cdot p_2$$

This constraint has two constraining points (p_1 and p_2) and actually limits two degrees of freedom of p_0 unless one chooses only one direction to calculate the ratio. The X and Y common directions would make **XRatio** and **YRatio** constraints. Also, in order to use a **Ratio** constraint with a single parameter, an additional assumption must be applied. For example, the most common assumption is that t+s=1, reducing the equation to:

$$p_0 = t \cdot p_1 + (1 - t) \cdot p_2$$

The parameter versions of all these relations link the numerical value of the constraint (the value v, distance d or ratio t) to a global font value that can be changed according to some global font conditions. Having many constraints use the same global parameter with the same value, promotes the conformity of shapes across the font.

The **Abs** constraint is mostly used for positioning points in some global parameter value (x-height or baseline) or to anchor points locally to the grid. It is usually used while rounding (either the global value is rounded or the point position is rounded to the closest grid line). The **Dist** constraint is used mainly to maintain feature widths such as stems, bars and also counters. Before applying the constraint, the width is usually rounded as well, so as to maintain global consistency. **Ratio** constraints are used to position points correctly between extreme points or other key-position points. The **Ratio** constraint is also used as the default constraint for all otherwise unconstrained points in order to preserve the continuous shape of the character.

The fact is that most hinting technologies move points or a group of points according to these three types of relations while grid-fitting the outline. In Table I we demonstrate the translation of these three relations to common directives in PostScript, TrueType, VisualTT and the FontLab font editor directives. This means that finding those types of relations between points in the outline is always necessary in any hinting process. However even if one would find all the 'correct' relations of such types

Table I. Point Relations and Their Closely Corresponding Commands in Various Hinting Languages

Relation	TrueType	PostScript	VTT	FontLab
Abs	MDAP	Ghost Stem	Anchor()	Align
PAbs	MIAP	Blue values	Anchor(cvt)	Align (top/bottom)
Dist	MDRP	Stem (H&V)	Link()	Link
PDist	MIRP	StdHW, StdVW	Link(cvt)	Link+connection
Ratio	IP	Interpolation	Interpolate()	Interpolate
PRatio	—	—	—	—

in a character (which may be a matter of personal preference in many cases) it would not be sufficient. Section 3.6 discusses the reasons for these difficulties.

3.3 Hinting Situations

The first stage of the automatic procedure for hinting is the extraction of typographic information from each character outline. This process is detailed in Shamir and Rappoport [1996] and illustrated in Figure 3. Points that lay on the outline of the curve are of much greater importance than control points that only define the spline curve shapes. Hence, feature extraction starts with the classification of outline on-contour points. These points are classified according to geometric properties such as extremum points, inflection points, tangent points, vertical corners, and horizontal corners. In cases where the outline of the character is poorly sampled or wrongly defined (for example, missing extremum points) some quality enhancement preprocessing is necessary (see Shamir and Rappoport [1998b]). There are around 20 different types of points identified (Figure 4). These types are then used for the identification of straight segments, curved segments of various types, and other particular character parts such as serifs.

All the segments found are sorted: horizontal segments by their y value, vertical segments by their x value and slanted segments by their angle. This is done in order to match overlapping pairs of segments to find features such as bars, stems, bows, arcs, and slants. Other features identified include the baseline segments, the top-line segments, shape peak segments and more.

Serifs and other complex parts are found using finite automata that characterize the sequence of typical points defining their outline shape. Although instances across the font of one type of serif could induce several different finite automata, the definition of all the different sequences can be performed by simply choosing the specific outline parts that define the sequence. Any instance of this particular sequence in any character in the font will automatically be detected and classified as the same type of serif (see example in Figure 8).

The key observation for automatic hinting is that very often the hinting situations are targeted at preserving the typographic feature shapes after grid-fitting the outline. As mentioned earlier, the basis for most hinting languages is point relations. Therefore, our scheme converts the features into hints by identifying the proper constraints imposed by each feature on the points composing it and coding them in the relevant hinting language. However, before we can convert the features into real constraints, global sizes such as heights and widths need to be identified in the font.

3.4 Global Font Sizes

An important part of typeface design is the definition of several characteristic sizes. These include the usage of specific widths in stems and bars that occur across the font (see Figure 5); it includes the usage of specific heights for lower-case and upper-case characters, for straight (X, F, H, T, B etc.) and round (O, C, Q, G etc.) characters, and more. In order to promote conformity in the font bitmaps any recurring size should be used as a parameter for the appropriate constraint defined for grid-fitting. For example,

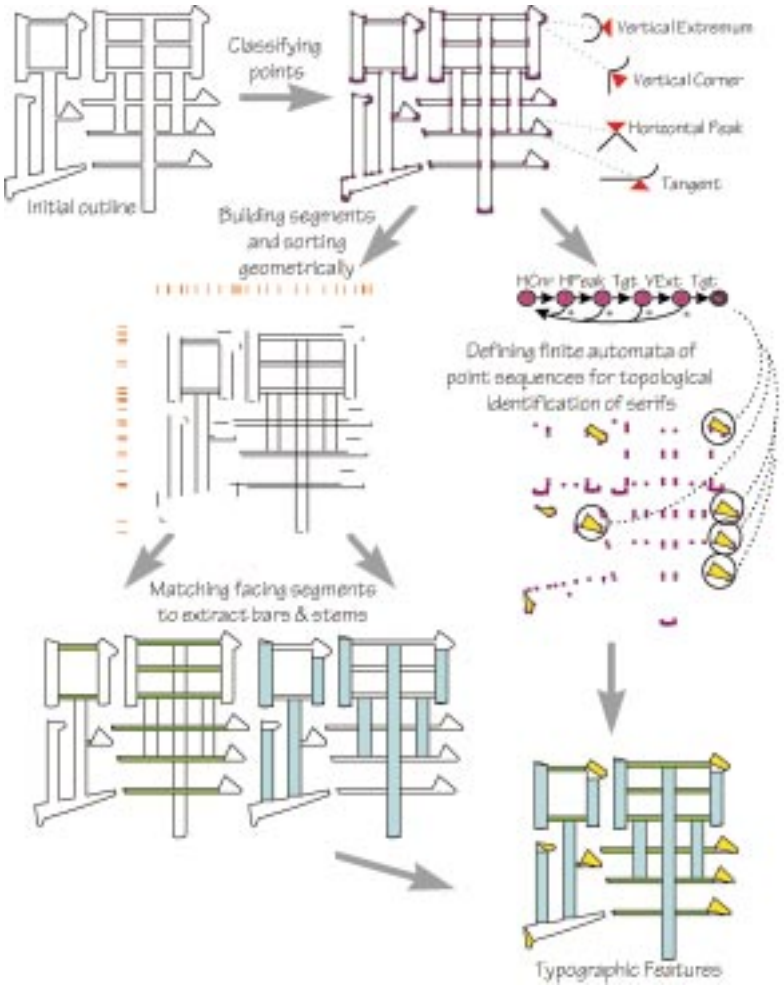


Fig. 3. The process of typographic feature extraction from outline representation starts with point classification according to their geometric properties. Straight and curved segments are extracted, sorted, and matched in order to find overlapping segments that create bars and stems. Using finite automata identifies more complex features like serifs by matching the sequence of outline points that defines them.

a characteristic width should be used as a parameter in a distance constraint created by a stem feature having this specific width, or a characteristic height should be used in an absolute value constraint imposed by some character having this height.

The use of these parameters can also be extended to constraints that include values that are only close (under some tolerance) to the characteristic value. This will promote further conformity and regularity in the font appearance. In fact, this mechanism is often used by hinting languages: several close sizes are snapped to a single size value at lower resolution, but maintain their differences at higher resolution.

While manually hinting a font, the font sizes must be collected and defined prior to any single character hinting. Specific characters should be measured and a table with such entries as x-height and stroke-widths should be compiled manually. This process can be done for most Roman style fonts.

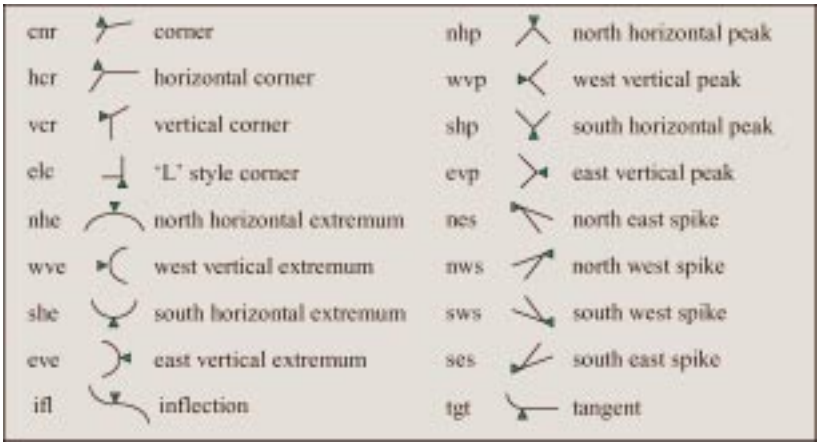


Fig. 4. Some examples of point classification on the outline of characters.



Fig. 5. Characteristic stems and bars in Times-Roman and Helvetica Bold fonts.

However, it is practically impossible to find all those values in Far-eastern fonts, where there are thousands of characters, and thousands of heights and widths values to measure (Figure 6).

Gathering global sizes automatically is done employing the features added to the outline at the previous stage. The sizes of each specific type of feature are measured and size count statistics are gathered: width of bars and stems, height of segments, height of characters and so on. For example, Table II shows the heights extracted automatically from the top and the bottom segments of horizontal features. All heights with more than 5 occurrences were stored in the table, and in fact, all principal heights characterizing the fonts were found automatically. The automatic process is less error prone and more objective: the bottom segments heights measured in Poor-Richard font range from −35 to −6, and in fact this font has several different baseline values for different characters (and none of them sitting on a zero-baseline). It would be much more difficult to find this fact manually and it would be wrong to choose a single baseline value.

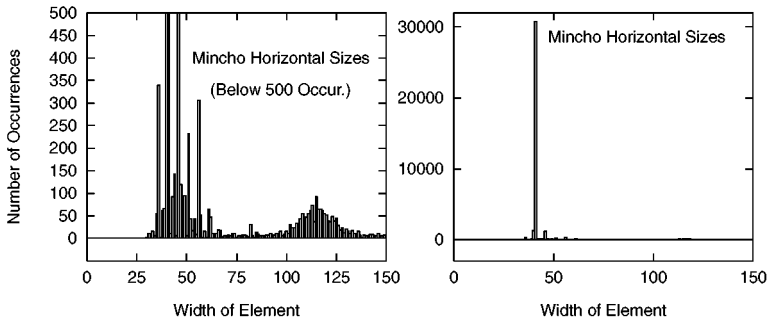


Fig. 6. Count of characteristic widths of bars in a Mincho style Japanese font. There are thousands of bar features measured; a process that is practically impossible to perform manually.

Table II.
Automatically extracted characteristic heights tables of Times-Roman and the bottom segments heights extracted for Poor-Richards font. All the important font heights were found and stored in the tables, even in irregular cases where no single baseline could be found (Poor-Richards font).

Times-Roman Top Segments Heights

value	Occurrences	Interpretation
36	8	Baseline bars
37	18	Baseline bars
916	9	Straight lowercase
943	20	Round lowercase
1356	22	Straight Uppercase
1384	9	Numerals height
1387	19	Round Uppercase
1422	11	h-stem height

Times-Roman Bottom Segments Heights

Value	Occurrences	Interpretation
−442	6	Lowercase descent
−31	9	Round uppercase
−28	23	Round lowercase
0	32	Baseline
1275	7	Uppercase top-bar
1319	17	Uppercase top-serif

Poor-Richard Bottom Segments Heights

Value	Occurrences	Interpretation
−35	8	Baseline
−27	6	Baseline
−20	17	Baseline
−16	9	Baseline
−14	6	Baseline
−6	8	Baseline

These tables are built automatically using the most frequent values of each size and storing their specific type: white or black distance, vertical or horizontal values, top or bottom heights, left or right segments. During the next stage, when converting the features to point relations, each constraint value is compared to the table entries of its corresponding type. If the actual value is close enough to some table value, the constraint is linked to the table value as its global parameter.

3.5 Constraint Graph Construction

At this stage the point constraints graph of each character is constructed iteratively by adding the constraints imposed by each typographic feature found in the first stage (Section 3.3). The points composing each feature are added to the graph as nodes (if they do not exist already), and the point relations are added as edges in the graph.

For example, the conversion of features such as bars, stems, arcs, bows and slants is done as follows. One side of the feature (usually the left or bottom side) is anchored using an **Abs** constraint on one of its

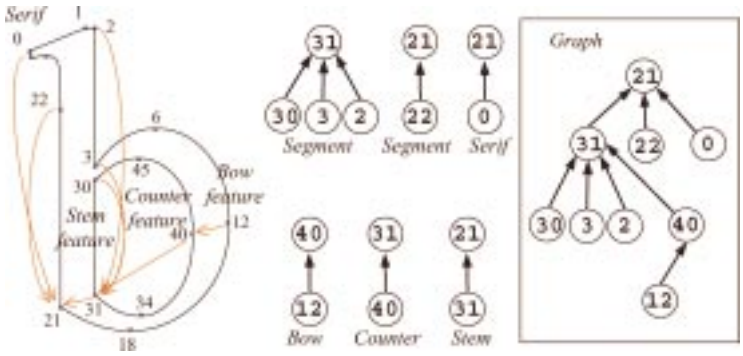


Fig. 7. Sub-graphs of constraints (middle) imposed by several features in lowercase 'b' character (left), and the graph constructed after topological sorting (right).



Fig. 8. Constraints defined on one serif can be carried over automatically to many serifs of the same type in all characters of the font.

points (usually the middle or the first point). All other points of this segment are then positioned relative to this first point using a **Dist** constraint (or if they are aligned, an align constraint, which is simply a zero-distance constraint). The other side of the feature is connected to the first side using either a **Dist** constraint or a **PDist** constraint, depending on the closeness of the value of the distance to a global font table distance. This is done by connecting one of the other segment's points to the first side point using the chosen constraint, and linking all other points with a **Dist** constraint to this point (Figure 7).

Segment features (also phantom segments such as for peaks in the outline) are converted to an **Abs** constraint on one of the points composing them. Other points are again linked by a distance constraint to the first point. If the absolute value is close enough to a global font value (for instance to a specific height), the constraint is linked to the global size as a **PAbs** constraint. Serif features or more complex features can only be set as declarative hints (for instance, in VTT there is a Serif() declaration) or they must have a predefined set of constraints. The predefined set of constraints must be inserted once for all such features in the font, and then it will automatically be inserted to any similar feature found (see Figure 8).

In Far-Eastern fonts where characters have many bars and stems, an additional feature called a *ladder* is defined (a simplified version of it is usually used in the Roman lowercase 'm', which in PostScript actually has a special hint). This feature includes several consecutive bars or stems with more or less the same length and the counters between them having the same width. This complex feature imposes a set of constraints that position one bar (stem) bottom side (left side) relative to the next bar (stem) bottom side (left side), hence preserving both the counter and the stem width.

3.6 Graph Resolution

After the hinting situations are found and converted to point relations we are often left with a very complex constraint graph. This graph should be solvable under the additional strict typography restrictions: it should always have a solution and it should have a single solution. However, just applying all

constraints in this graph one by one may fail due to over-constrained points or circular constraints. As an example, the 'm' character has three vertical stems, which are converted to **PDist** constraints of the two segments on each side. It also has two counters between the three stems, which constrain the distance between the stems. Hence, the position of the rightmost segment of the right stem is determined in the x-direction by the sum of the widths of the three stems and the distances between the stems. However, this segment also represents the rightmost part of the letter (at least in sans-serif typefaces), and is therefore constrained by the total character width. Such over-constraining must be resolved, usually by giving preference to the total character width and the stem width constraints, sacrificing the counter widths (for a more detailed discussion of this see Stamm [1998a]).

Even without cycles or over-constraining, the order of applying the constraints is important. For example, preserving the width of a stem feature by moving its left side to be a specific distance from the right side, must be done only after the right side has finished moving (anchored or rounded to the grid line). In order to create coherent and applicable hints from the constraint graph the inconsistencies in the graph must be found and resolved, and the constraints must be ordered. Therefore, graph resolution includes three main sub-tasks:

—Identifying the order of dependency among points and preserving it while converting to hints. This is performed mainly by topological sorting of the nodes.

—Finding and removing over-constrained situations. This is performed by using priorities on the different constraints.

—Checking for cycles of constraints in the graph and removing them. This is done using the two previous methods: sorting helps find the cycles and priorities help break them.

To simplify these tasks even further, we actually work on a simpler sub-graph of constraints. Most features can be defined in terms of a number of segments, which in turn hold a set of points. For example the baseline is a segment, and bars and stems include two parallel segments each. Instead of inserting all constraints of all points into the graph, we choose a representative point for each segment, and insert it into the graph along with its dominant constraint. For example, an **Abs** constraint for a free segment, a **PAbs** constraint for a segment linked to a global font height, a **Dist** constraint for a ladder segment, and a **PDist** constraint for the dependent-side segment of a stem or a bar. All other segment points are linked later with a distance (or align) constraint to the representative point.

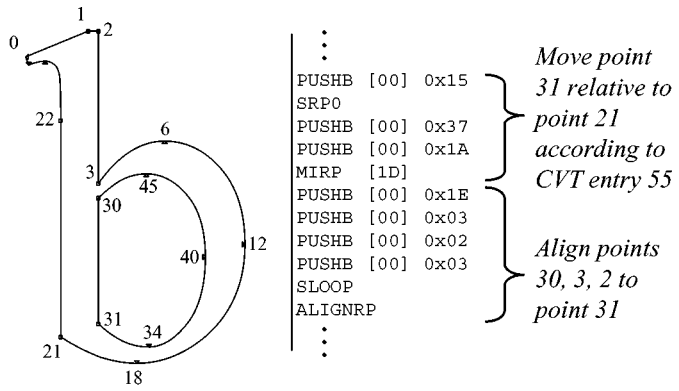
The resolution algorithm is actually a topological sorting of the nodes of the graph, with a pre-processing stage of removing over-constraints. Since each node represents a point, it can only have two constraints of the types we specified. We go over the nodes and delete lower priority constraints in any node with more than two constraints. Then we use the in-degree of the nodes—the number of nodes dependent upon it—for topological sorting. We use a queue and remove the nodes with zero in-degree, then remove their outgoing edges from the graph and so on. Each node removed is assigned an id with increasing order, hence after sorting we are left with a total order on the nodes. If we are left with cycles, we use the same priorities as in the pre-processing stage to break them. Therefore, the basis for cycle breaking and over-constraint removal is the priorities defined on the constraints. These priorities follow a simple set of rules:

(1) Font constraints (e.g. parametric constraints linked to a font global value) have priority over character constraints (non-parametric constraints).

(2) Character metrics constraints (baseline, character-width, character-height) have priority over local character constraints.

(3) Distance and ratio constraints have priority over absolute value constraint.

(4) When no other rule applies, choose arbitrarily and issue a warning.




```
...
NPUSHB (07) 1E 03 02 03 37 1A 15
SRP0
MIRP [1D]
SLOOP
ALIGNRP
...
```

Fig. 9. Part of a TrueType program created from a horizontal **PDist** constraint (top, compare with Figure 7), and the optimized version of the code (bottom).

In practice, the number of over-constraints or cycles that use the last rule is negligible, but it must be added if we would like a fully automatic hinting mechanism.

4. HINTING TRANSLATIONS

4.1 TrueType Instructions

After resolving the constraint graph, we have a coherent and ordered set of constraints for the nodes. We are now left with the task of converting this set to hints in the desired hinting language. In order to create TrueType hinting programs, we first convert the global font sizes table to the TrueType control-value table (CVT). We then convert each constraint to a small set of TrueType instructions, using the entries in the CVT if needed. The order of constraints imposes the order of instructions and assures the dependencies will be preserved.

For example, the code segment in Figure 9 shows part of a TrueType program created automatically to hint the lowercase 'b' character. The instructions shown are created from the **PDist** dependency of a segment inside a stem feature. The TrueType language is a stack based language, therefore the operands are pushed onto the stack before issuing a command. In post processing, the push instructions in a character program are moved to the front, and all other instructions are put at the end. By defining all small code segments generated from each constraint similar to the one in Figure 9, we can optimize the number of instructions in the character program.

4.2 PostScript Hint Sets

In PostScript global font values such as horizontal and vertical standard stem widths and blue-values (font heights) can be declared using the statistics gathered in the second stage. The declarative style of PostScript character hints can be created directly from the typographic features. Nevertheless, PostScript hints present their own difficulty. While drawing the outline for rasterization, the PostScript rasterizer uses a set of active horizontal and vertical stem hints to position the points of the outline.

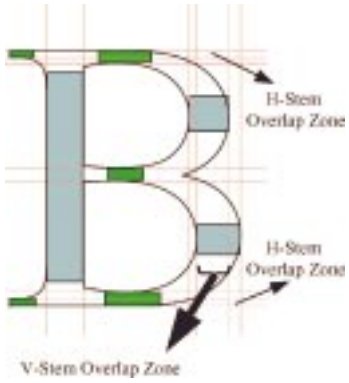


Fig. 10. Overlapping of stem hints in PostScript must be resolved by changing hint sets.

Define the *zone of influence* of each stem as any coordinate that falls between its two sides. In PostScript, the zone of influence of any two stems in the same set must not overlap. Unfortunately, there are many cases when stems do overlap (see Figure 10). This is especially true in Far-eastern characters where the number of vertical and horizontal stems in each character is large. Therefore, overlapping stems must be separated into different sets of hints, and the sets must be active in different places along the outline of the character. PostScript includes a hint that can change the set of active stems (vertical and horizontal). This hint is attached to a point on the outline, where the set should change while drawing the character outline. The most difficult problem while creating PostScript hints is breaking the horizontal and vertical stem hints of each character into active and non-overlapping sets.

Denote $\mathcal{P}$ as the set of points in a character, $\mathcal{G}$ the set of stems, $p \in \mathcal{P}$ is a point on the outline, and $g \in \mathcal{G}$ a stem defined in the character. We say that $p \in g$ if p is a part of the definition of g, and define a function $G : \mathcal{P} \rightarrow \mathcal{G}$, where $G(p) = g$ for all $p \in g$, otherwise $G(p) = \perp$ (on the rare occasion that p is included in more than one stem, we choose for example the widest stem as $G(p)$). We define a geometric test function $O : \mathcal{G} \times \mathcal{G} \rightarrow \{\text{true, false}\}$ that returns true if two stems have overlapping influence zone. Denote also S as any set of stems defined in a character ($S \subset \mathcal{G}$) and Σ as the set of all stem-sets in a character. A solution to the stem-set problem is a definition of a set Σ and a function $A : \mathcal{P} \rightarrow \Sigma$, which assigns one and only one set from Σ to a point $p \in \mathcal{P}$, and guarantees two conditions:

(1) Each set of stems must not contain any stems with overlapping influence zone:

$$\text{if } g_1 \in S \text{ and } g_2 \in S \text{ then } O(g_1, g_2) = \text{false}$$

(2) Each stem must be in the active set of stems for all the points composing it:

$$\text{if } p \in g \text{ and } g \in S \text{ then } A(p) = S$$

Since replacing the hints is both time consuming while rasterizing and space consuming in the font, the number of different sets ($|\Sigma|$) should be minimal, and the best possible position on the outline-contours to place the set replacements should be found. Instead of solving the full optimization problems, which might be time consuming, we resort to a simple greedy algorithm (Figure 12), which guarantees that the stems will be active in the places they should, and no overlap will occur. In practice, even in the most complex characters the number of set-replacements is still very low (see Figure 11). We chose to assign sets to the points in order since it is the order in which they are rasterized. Also, we chose to loop on the contours and fill the non-overlapping stems that touch this contour first inside

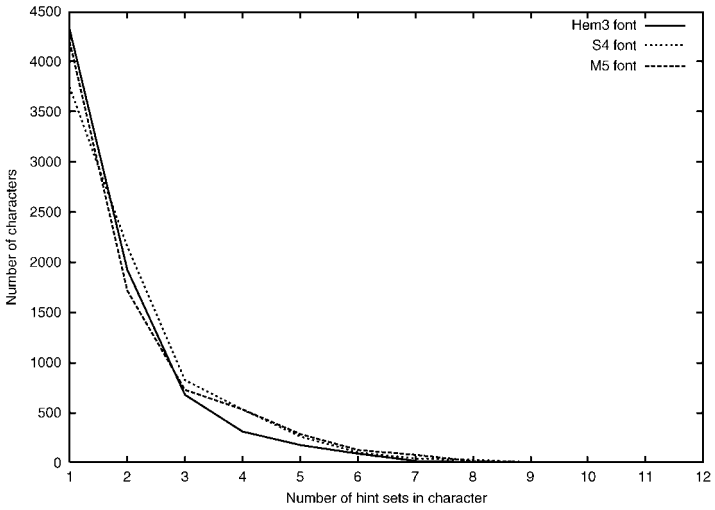


Fig. 11. The number of hint-set replacements remains very low even in Far-Eastern typefaces where characters are much more complex and may include many stems. More than 99% have less than 10 set-replacements and the average number of sets in a character is below 2.

```
For each point p ∈ P
   Set A(p) = ⊥
For each stem g ∈ G and each p ∈ g
   Set G(p) = g
Create S = ∅
For each contour of the outline do {
   Loop on points p in contour {
      If G(p) = ⊥ or G(p) ∈ S
         A(p) = S
         Continue (to next point in contour)
      Set g = G(p)
      If ∀g' ∈ S, O(g',g) = false
         S = S ∪ {g}
      Else
         Create a new current set S = {g}
         Fill S with all non-overlapping
            stems touching this contour
      A(p) = S
   }
}
For each set S ∈ Σ and each g ∈ G
   If ∀g' ∈ S, O(g',g) = false
      S = S ∪ {g}
```

Fig. 12. The algorithm for partitioning the whole group of stems into non-overlapping sets.

the loop, since they are more important for the rasterization of this contour and should be included before any other (possibly overlapping) stem. The algorithm runs in $O(|\mathcal{P}| \times |\mathcal{G}|)$. It takes $O(|\mathcal{P}| \times |\mathcal{G}|)$ to calculate the function G, by looping on all $g \in \mathcal{G}$ and for each point $p \in g$ setting $G(p) = g$. The main loop goes over all contours and for each contour loops on its points, hence each point is visited once. For each point p the worst case for checking if the stem $G(p)$ overlaps any other stem is at most $O(|\mathcal{G}|)$. Expanding the sets takes $O(|\mathcal{G}|^2)$, but $|\mathcal{G}| < |\mathcal{P}|$, hence we are left with $O(|\mathcal{P}| \times |\mathcal{G}|)$. The algorithm

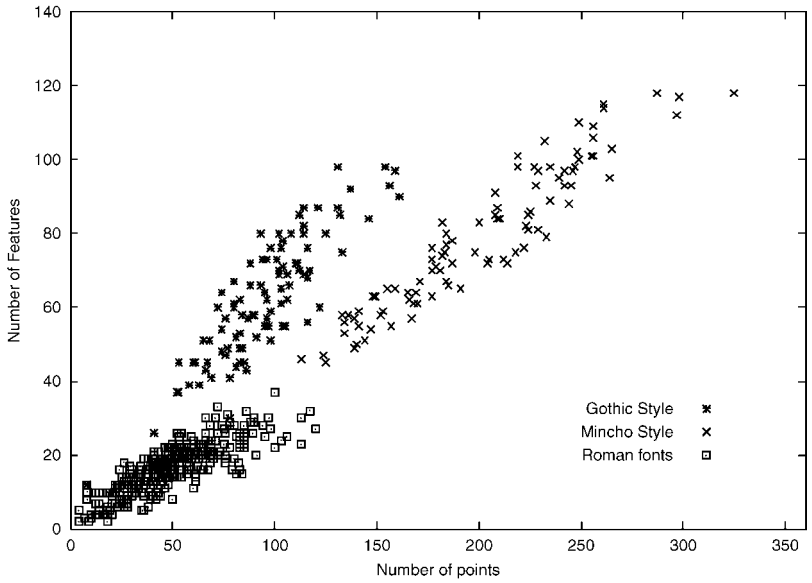


Fig. 13. The number of typographic features extracted from a character is proportional to the number of points (each plotted-point represents one character). The proportion between them changes with the style of the font. The relatively high number of features counted comes from the fact that we define all sub-parts of every compound typographic feature as a feature by itself. For example, A ladder feature will include many stems and counters, which in turn will include many more segments.

guarantees condition 1 since for each point, its stem will be in the active set either by expanding a set or by creating a new set that includes it. Condition 2 is guaranteed by always checking for overlapping before adding any stem to a set.

5. IMPLEMENTATION AND RESULTS

The time complexity of the different stages of the algorithm varies. Denote n as the number of points in a character and m as the number of features. Statistic gathering, constraint graph construction, graph resolution with topological sorting and hint translation stages are all linear in the number of features, meaning $O(m)$. The heaviest task is that of feature extraction in the first stage. Feature extraction includes point classification, which is $O(n)$, segments sorting and matching, which is $O(m^2)$, serif and higher level features identifications, which are $O(n \times m)$. Therefore, the overall complexity remains $O(m \times n)$. Since the number of features is proportional to the number of points (see Figure 13), this can be seen as almost quadratic in the number of points—$O(n^2)$. However, the maximum number of points in one character is not more than 350 and the actual running time is extremely fast. On a pentium-III 700 mhz with 128 mb RAM it takes less than 30 seconds to automatically hint any roman font, and up to 5 minutes to hint any Far-Eastern font (with up to 8,000 characters).

Since the TrueType program or PostScript instructions are not very informative in terms of illustrating hinting results, we chose to show (Figure 14) a snapshot of automatically created VTT hints for a similar character in several fonts (Times, Poor-Richard, Schoolbook, Georgia and Courier). These examples present several different topologies of the same character, which would be difficult to capture using example-based hinting methods such as Zongker et al. [2000] and Hersch and Bétrisey [1991]. In our method, no previous model or previous manual hinted font was used in order to produce the hints shown (see also Figure 15 for more complex automatic hinted characters).



Fig. 14. The automatically hinted 'R' character from several different typefaces. The hints shown in VTT are only the main part of the whole set of hints for aligning and positioning points relative to the ones shown. Also, the links to CVT were dropped.

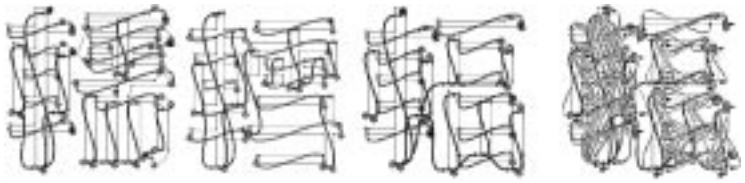


Fig. 15. Several automatically hinted characters from a Mincho style Japanese font. Only the main point dependencies are shown in the left three characters. All point relations and hints are illustrated in the rightmost character.



Fig. 16. Times (left) and Georgia (right) Roman fonts bitmaps created without hinting and no dropout control (top), compared to automatic hinting (middle) and manual expert hinting (bottom).

After converting the hints to TrueType programs and creating a font, we illustrate the difference between un-hinted, automatically hinted and manually hinted characters in Figure 16. It is well known that at small point sizes the continuity of rasterized strokes is often ensured simply by using dropout control (see Hersch [1993]). Nevertheless, other important factors such as width uniformity of typographic features (stems, arcs), relative feature and character sizes and an overall font harmonic view are only preserved using hinting. Using our auto-hinting method these are illustrated in Far-eastern fonts (Figure 17), and in different point sizes (Figures 18 to 20).

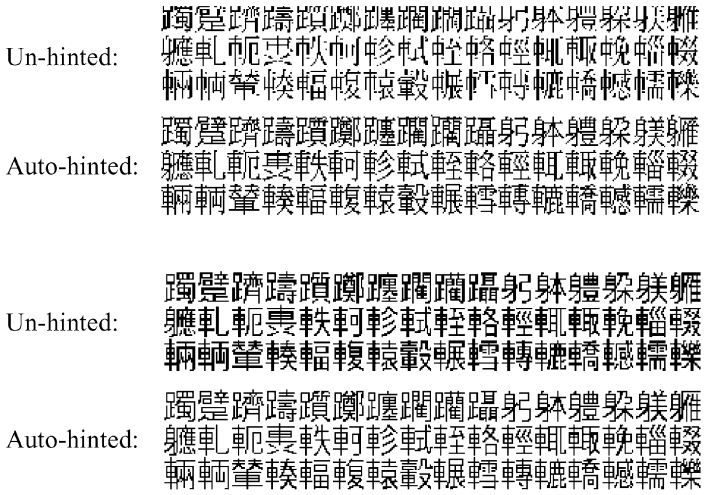


Fig. 17. Bitmaps created without hinting Vs. bitmaps created with automatic hinting of Far-eastern fonts. The top two samples are from a Japanese Mincho style font and the bottom two from a Japanese square gothic style font. The bitmap size is 23 ppem (24 points at 120 dpi).



Fig. 18. Cascading sizes of Courier typeface (top) and Times-Roman (bottom) hinted automatically. Note the uniformity of characters for a given size and the move from one pixel width per feature to two pixels almost consistently in the font (between 28 to 29 in Courier and between 13 to 14 in Times).

6. SUMMARY AND FUTURE DIRECTIONS

We have presented a fully automatic method for the creation of rasterization hints for digital typefaces. The method combines local character and global font considerations and relieves the user from most



Fig. 19. Japanese Square-Gothic style font automatically hinted in different sizes.



Fig. 20. Japanese Mincho style font automatically hinted in different sizes.

tedious and time-consuming tasks. It can support hinting of fonts in any language and any type of hinting technique (PostScript and TrueType). Although quadratic in nature, the actual hinting running time is only minutes even for fonts containing thousands of characters. The rasterization of the resulting auto-hinted fonts produces high-quality bitmaps, and may also be used as a basis by means of which further manual hinting refinement may be done much more rapidly using tools such as the Visual TrueType editor.

The basis on which our method is built is feature-extraction. In this technique typographic information is restored from the outline low-level representation and used both in global font statistics and local character hinting. Instead of using feature extraction, a similar but more powerful technique is to use feature-based design. In feature-based design, the designer incorporates high-level knowledge and typographic characteristics into the model while designing it. This may include designation of features, of typographic constraints that need to be preserved and parameters that may vary and change the design. Such a design scheme carries many advantages (see Shamir and Rappoport [1998a]), one of which

is to greatly simplify the task of automated hinting. For example, the same typographic constraints that are part of the model definition will be used during rasterization. Furthermore, there will be no need to convert all constraints to point relations so higher-level and more declarative definitions can be used. Such advanced font models are yet to be realized and adopted by technology leaders such as Microsoft and Adobe. However, the gains of using such models will be evident especially in automatic rasterization and hinting.

ACKNOWLEDGMENT

I am grateful to Heshev Development L.T.D. for providing access to their systems and typeface data for this research.

REFERENCES

ADOBE SYSTEMS INC. 1990. *The type 1 format specification*. Addison Wesley.

ADOBE SYSTEMS INC. 2000. *OpenType specifications*. http://partners.adobe.com/asn/developer/opentype/main.html.

ANDLER, S. 1990. Automatic generation of gridfitting hints for rasterization of outline fonts or graphics. In *EP90, Proceedings of the International Conference on Electronic Publishing, Document Manipulation & Typography*, R. Furuta, Ed. Cambridge University Press, 221–234.

BÉTRISEY, C. AND HERSCH, R. 1989. Flexible application of outline grid constraints. In *Raster Imaging and Digital Typography*, J. André and R. Hersch, Eds. Cambridge University Press, 242–250.

BOUMA, W., FUDOS, I., HOFFMANN, C., CAI, J., AND PAIGE, R. 1995. Geometric constraint solver. *Computer-Aided Design 27*, 6, 487–501.

HERSCH, R. 1993. Font rasterization, the state of the art. In *Visual and Technical Aspects of Type*, R. Hersch, Ed. Cambridge University Press, 78–109.

HERSCH, R. AND BÉTRISEY, C. 1991. Model based matching and hinting of fonts. In *ACM Comput. Graph., Proceedings SIGGRAPH'91*. Vol. 25. Addison Wesley, 71–80.

HERTZ, J. AND HERSCH, R. 1994. Towards a universal auto-hinting system for typographic shapes. *Electro. Publ. 7*, 4, 251–260.

KAROW, P. 1989. Automatic hinting for intelligent font scaling. In *Raster Imaging and Digital Typography*, J. André and R. Hersch, Eds. Cambridge University Press, 232–241.

KAROW, P. 1994. *Font Technology: Description and Tools*. Springer-Verlag, Berlin.

KRAMER, G. 1992. *Solving Geometric Constraint Systems*. MIT Press.

MICROSOFT CORPORATION. 1990. The TrueType font format specification.

MICROSOFT CORPORATION. 2000. *Visual TrueType Page, software description*. http://www.microsoft.com/typography/tools/vtt.htm.

MICROSOFT CORPORATION. 2001. *OpenType initiative FAQ*. http://www.microsoft.com/typography/faq/faq9.htm.

PYRUS NORTH AMERICA. 2001. *FontLab User Manual*. http://www.font.to/.

SHAMIR, A. AND RAPPOPORT, A. 1996. Extraction of typographic elements from outline representations of fonts. *Comput. Graph. Forum 15*, 3, 259–268. Proceedings of Eurographics '96.

SHAMIR, A. AND RAPPOPORT, A. 1998a. Feature-based design of fonts using constraints. In *Electroninc publishing, Artistic Imaging, and Digital Typography, LNCS, Proccedings RIDT'98*, R. D. Hersch and J. André, Eds. Springer-Verlag, 93–108.

SHAMIR, A. AND RAPPOPORT, A. 1998b. Quality enhancements of digital outline fonts. *Comput. Graph. 21*, 6, 713–725. Special issue on Graphics in Electronic Printing and Publishing.

STAMM, B. 1998a. *The raster tragedy at low resolution, or: why correct math looks wrong on screen and how to fix it*. http://www.microsoft.com/typography/tools/trtalr.htm. Excerpts from the presentation at the training session on 29 October 1997 (updated in March 1998).

STAMM, B. 1998b. Visual truetype: A graphical method for authoring font intelligence. In *Electronic publishing, Artistic Imaging, and Digital Typography, LNCS, Proccedings RIDT'98*, R. D. Hersch and J. André, Eds. Springer-Verlag, 77–92.

ZONGKER, D., WADE, G., AND SALESIN, D. 2000. Example-based hinting of truetype fonts. In *ACM Comput. Graph. Proceedings Siggraph'00*. Addison Wesley, 411–416.

Received December 2001; revised August 2002; accepted September 2002